

— BACKEND ENGINEER · COMMERCE DOMAIN

AI-Driven Development Lifecycle 관점으로 시스템을 개발하는 김찬영

커머스 복지물의 주문·배송·결제 도메인을 전담하며, **MSA 분산 트랜잭션**과 **비동기 메시징**으로 반복 장애를 줄이고 정책 변경에 강한 구조를 설계해 왔습니다.



CHAN YEONG · 1993

EMAIL

emrhssla@gmail.com

PHONE

010-9192-1372

LOCATION

경기 수원시 권선구

PORTFOLIO

PORTFOLIO

[chanyoung-dev.github.io/Profile/portfolio](#) ↗

4.1 YRS

COMMERCE BACKEND

MSA

SAGA · DISTRIBUTED
TRANSACTION

AI-DLC

AI-DRIVEN
DEVELOPMENT

TEST CODE

TESTCONTAINERS ·
TDD

01 간략 소개

PROFILE

정보통신공학을 전공하고 커머스 핵심 도메인에서 실무 경험을 쌓아온 개발자입니다. 제가 개발자로서 전문성을 쌓아온 경험은 크게 두 가지입니다.

첫째, 고객사 맞춤 복지물 서비스를 제공하는 회사에서 **주문·결제 도메인**을 전담하며 실무를 수행했습니다. 고객사별 맞춤 정책과 복지포인트, 다양한 결제·배송 규칙이 공존하는 환경에서 정확성과 신뢰성이 중요한 핵심 도메인을 **MSA와 분산 트랜잭션 구조**로 운영하며 안정성을 높여왔습니다. 일일 모니터링으로 오류 여부를 상시 점검하고, 단순 조치에 그치지 않고 객체지향적 리팩토링, 테스트 코드, MQ 비동기 처리를 통해 근본 원인을 제거하며 신뢰성과 효율성을 높였습니다. 나아가 **Harness Engineering**을 통해 안전한 운영 시스템 접근 및 실무형 AI 개발 체계를 구축하여 AI 생산성을 크게 향상시켰고, **AI-DLC 개발 방법론**을 도입하여 개발 공수 기간을 대폭 감소시켰습니다.

둘째, 개인 프로젝트로 **AI-DLC 개발론**을 통해 코인 자동매매 및 알림 서비스를 직접 설계·개발·운영한 경험이 있습니다. 실자동매매 시스템, 알림 서비스와 백테스트 환경을 구축하였고 AI서비스를 연동한 AI 포트폴리오 대시보드를 개발했습니다. 이 경험을 통해 멀티프로세스 기반의 대규모 데이터 처리 경험을 쌓을 수 있었습니다.

앞으로는 이러한 경험을 바탕으로 **신뢰성 높은 시스템 구축**과 **대규모 트래픽에도 문제없는 시스템**을 만들겠습니다.

02 학력 / 교육

EDUCATION

성경대학교 (4년제) · 정보통신공학

2013.03 — 2020.02 졸업 학점 3.17 / 4.5

영생고등학교 · 이과계열

2009.03 — 2012.02 졸업

구름 · 쿠버네티스 전문가 양성 과정

2021.12 — 2022.05 네트워크 · 리눅스 · 도커 · AWS · 쿠버네티스

기술 블로그 운영

2018.12 — chanyoung-dev.github.io

03 자격 / 어학 / 수상

CERTIFICATIONS

정보처리기사

한국산업인력공단 2019.08

TOEIC 760

영어 2020.07

한이음 ICT 공모전 입상

한국정보산업연합회 2019.12

한국사능력검정시험 1급

국사편찬위원회 2019.08

SK Supex

SK 2025.03

MOS Word 2010 Expert

Microsoft 2018.10

MOS Outlook 2010 Core

Microsoft 2018.12

1종 보통 운전면허

경찰청 2014.10

04 기술 스택

SKILLS

BACKEND

Java Spring Boot Spring Framework
Spring Data JPA MyBatis JSP
Thymeleaf MSA RabbitMQ
Python Node.js

FRONTEND

JavaScript HTML/CSS jQuery
Vue.js React

INFRA · DB · ETC.

Oracle SQL Docker Jenkins
AWS Kubernetes CI/CD git
AI-DLC 개발론

05 경험 요약

EXPERIENCE SUMMARY

✓ **도메인 전문성** — 핵심 커머스 로직(배송비·결제·취소·교환·반품) 깊은 이해

✓ **테스트·자동화 안정성** — 검증 체계 도입으로 장애율 대폭 감소

✓ **아키텍처 설계** — 유지보수 비용 절감, 확장성에 강한 설계

✓ **대규모 트래픽 + MQ** — 비동기 메시지 큐 처리 경험

✓ **MSA · 분산 트랜잭션** — 주문·결제·배송 도메인 실전 경험

✓ **AI 활용** — AI-DLC 개발론 및 Harness 적용

SK 엠앤서비스(주) · 커머스개발1팀

2022.07 - 재직중

대리 / 팀원 4년차 · 풀스택 개발자

설립일 2000.02 주요업종 시스템 SW 개발/공급 매출 2,470억 임직원 1,132명

PROJECT 01

2026.06 - 2026.10

PG 고도화 프로젝트

Spring Boot

Java

JavaScript

MyBatis

Oracle

MSA

RabbitMQ

Docker

Jenkins

결제 · 현금영수증 PG사 전환 프로젝트.

현금영수증 발급 RabbitMQ 도입 및 발급 구조 개선을 AI-DLC 기법 기반의 AI 주도 통합 개발로 수행.

WHY 문제 상황

- 모든 현금영수증이 자사 명의로 일괄 발급 → 실제 매출 주체 반영 어려움
- 예외 케이스로 인한 운영 수기 처리 발생
- 동기 방식 현금영수증 처리 → 응답속도 지연
- 현금영수증 처리 실패가 주문 실패로 장애전파
- 대규모 트래픽 시 서버 과부하로 인한 다운 위험

HOW 해결 과정 및 임팩트

- 다중 벤더 주문은 벤더별 금액 · 면세금액 분리 후 고유 주문번호로 개별 발급 · 취소 처리
 - 실제 매출 귀속 주체에 맞는 벤더별 발급 지원
 - 매입 · 위탁 등 거래유형에 따라 실제 사업자번호로 발급
- 현금영수증 관련 어드민 기능화 → 운영 수기 처리 최소화
- 주문 완료 이후 현금영수증 처리를 RabbitMQ 기반 비동기 방식 으로 분리, 재시도 큐와 DLQ를 적용해 오류 제어
 - 응답속도 개선 및 장애 격리, 대규모 트래픽에도 대응 가능한 확장성 · 가용성 확보
- 컨테이너 및 오케스트레이터 등 인프라 생명주기 제어를 통해 메시지 무손실 보장
- 동시성 제어
 - 1차 — Single Consumer로 메시지 직렬화하여 순서제어
 - 2차 — Consumer 수평 확장 + 낙관적 락
- AI-DLC 개발론을 활용해 개발 공수(MM) 66% 절감

PROJECT 02

2024.12 - 2025.06

주문 고도화 프로젝트

Spring Boot

Java

MyBatis

Oracle

MSA

RabbitMQ

Docker

Jenkins

주문 관련 대규모 커머스 리뉴얼 프로젝트. 상품·배송비 구조 분리, 취소/교환/반품 개선, 주문 시스템 안정화.

문 제 상 황

- 상품·배송비·주문이 강하게 결합 → 정책 변경 어려움
- 테스트 부재로 정책 변경 시 대규모 수동 검증 필요
- 사이드 이펙트 우려로 개선 속도 저하

최 종 임 패 크

- MTTR 45분 → 12분 (73% 단축)
- 정책 변경에 유연한 구조 확보
- 복잡 케이스(조건부·부분 취소) 유연 대응

2.1 MSA 주문·배송·결제 전면 재설계

- 상품·배송비 구조 N:N 분리, 취소/반품/교환 프로세스 추가·개선
- 멍등성 · 동시성 · 분산 트랜잭션 제어, 변화에 강한 객체지향 리팩토링
- 배송비만 취소, 귀책 전환 환불, 조건부 무료배송 부분 취소 등 복잡 케이스 처리

2.2 핵심 시나리오 중심 테스트 코드 도입

- MSA 통합 테스트 환경에 대한 이해 → 주문·결제·배송 핵심 시나리오 테스트 작성
- Persistence layer 테스트환경 구축
- 주문 관련 장애 평균 복구 시간(MTTR) 45분 → 12분 (약 73% 단축)

어 려 웠 던 점

- persistence layer 환경구축 문제 → 테스트 이후의 개발 DB 오염, 분산 트랜잭션 롤백 필요
- MSA에서의 DB 공유로 인한 타 API mocking 실패
- 선행 데이터 구축 → 주문테스트 진행을 위해선 상품데이터가 필요

2.3 운영 모니터링 및 운영 개선

- 정기 일일 로그 분석으로 장애 발생 여부 점검, 오류 원인 파악 및 개선 조치를 통한 운영 안정화
- 실시간 모니터링(DataDog) 기반 상시 감시, 정책·비즈니스 로직 수정으로 지속적 오류 개선
- 재고 관련 동시성 처리, 포인트 연마감 결제취소 이슈 등 주요 오류 근본 원인 분석 및 개선
- 초기 대비 장애율 76.5% 감소

PROJECT 03

2026.05 — 재직중

AI Tech TF · AI 도입 리드

정책·코드·운영 데이터를 잇는 AI 아키텍처 설계와 Claude Code Harness Engineering으로 사내 AI 개발 체계 구축.

WHY

- 정책·코드·운영 데이터가 분리되어 AI가 문맥을 연결하지 못함 (Context Fragmentation)
- 아키텍처·정책·비용·보안 등 구조를 모르기 때문에 AI 환각 발생, 컨텍스트·토큰·응답시간 낭비
- 보안상 운영 DB 직접 접근 불가 → LLM의 업무 처리 범위 축소, 부정확한 개발 진행
- AI 산출물이 통일되지 않은 코드 컨벤션으로 개발됨

IMPACT

- AI 아키텍처 설계로 외부 LLM 활용 시에도 운영 원본 데이터 유출 차단 구조 확보
- Harness Engineering 기반으로 정책·코드·운영 데이터를 AI가 한 번에 연결해 운영 이슈 분석 속도 향상
- AI-Driven Development Lifecycle 관점 개발을 통해 프로젝트 개발 공수 MM 대폭 감소
- Claude Code 환경 구축으로 AI 산출물 품질 향상·통일화 및 신규 합류자 도메인 진입 비용 감소
- Harness Engineering 기반으로 AI 환각 차단, 컨텍스트·토큰·응답시간 효율적 활용

PROJECT 04

2025.06 — 2025.07

ISMS 대응 — XSS 통합

MSA 구조에서 서비스별로 제각각이던 XSS 대응 방식을 전사 공통 정책으로 통합.

- 케이스별 XSS 처리 기준 통합 및 가이드 문서화
 - 전사 공통 XSS 애노테이션 방식으로 통합 적용
 - 이중 escape 처리 / JSON 파싱 오류 재발 방지
- 다음 해 ISMS 점검에서 XSS 관련 80% 단축

코인 자동매매 및 알림 서비스

2024.01 - 2025.10

설계 · 개발 · 운영 단독 수행 · 기여도 100%

Node.js

Python

React

Express

MySQL

Binance API

Upbit API

TradingView

코인 시장 실시간 데이터 변동성을 분석해 **지표 개발** → **백테스트** → **자동매매** → **알림 서비스화**까지 전 과정을 직접 수행. 멀티프로세스 백테스트와 자동화 기술로 안정적·정합성 높은 매매 시스템 구축.

① 지표 개발

- 거래량 폭등 · 추세 반전 캔들패턴 기반 **역추세 전략 지표** 설계 및 구현
- TradingView Script 시각화, 조건 충족 시 **자동 알림** 기능

② 백테스트 및 전략 검증

- 수백만 케이스를 **멀티프로세스 병렬 처리** — 단일 프로세스 대비 **약 10배 향상**
- 상승률 기울기, 최대낙폭(MDD), 변동성 지수(Volatility), PSI 등 다양한 메트릭으로 전략 평가
- 재현성 약 **99% 확보**

③ 자동매매 시스템 구축

- 백테스트 검증된 전략을 **Binance API 자동매매**로 구현, 실매매 환경 수익률·안정성 검증
- **이중화 / 장애감지 / 장애예방** 메커니즘 설계로 가용성 향상

④ 급등락 알림 서비스

- Binance 데이터 실시간 모니터링, 급등·급락 발생 시 **SNS-메신저 자동 알림**
 - Telegram · X · Instagram · Facebook · Threads **멀티 플랫폼 동시 포스팅**
 - Upbit API 연동, 국내 거래소 가격·티커 한글 제공
- **2025.12 기준 Telegram 구독자 830명**

⑤ AI 포트폴리오 대시보드

- Claude Code(**AI-DLC 개발론**) + React로 코인 수익 포트폴리오 대시보드 개발
- AI 서비스 연동, 단순 개발을 넘어 **데이터 분석 · 전략 설계 · 서비스화 전 과정 독립 수행**